

Assignment 3 | Bridge Pattern

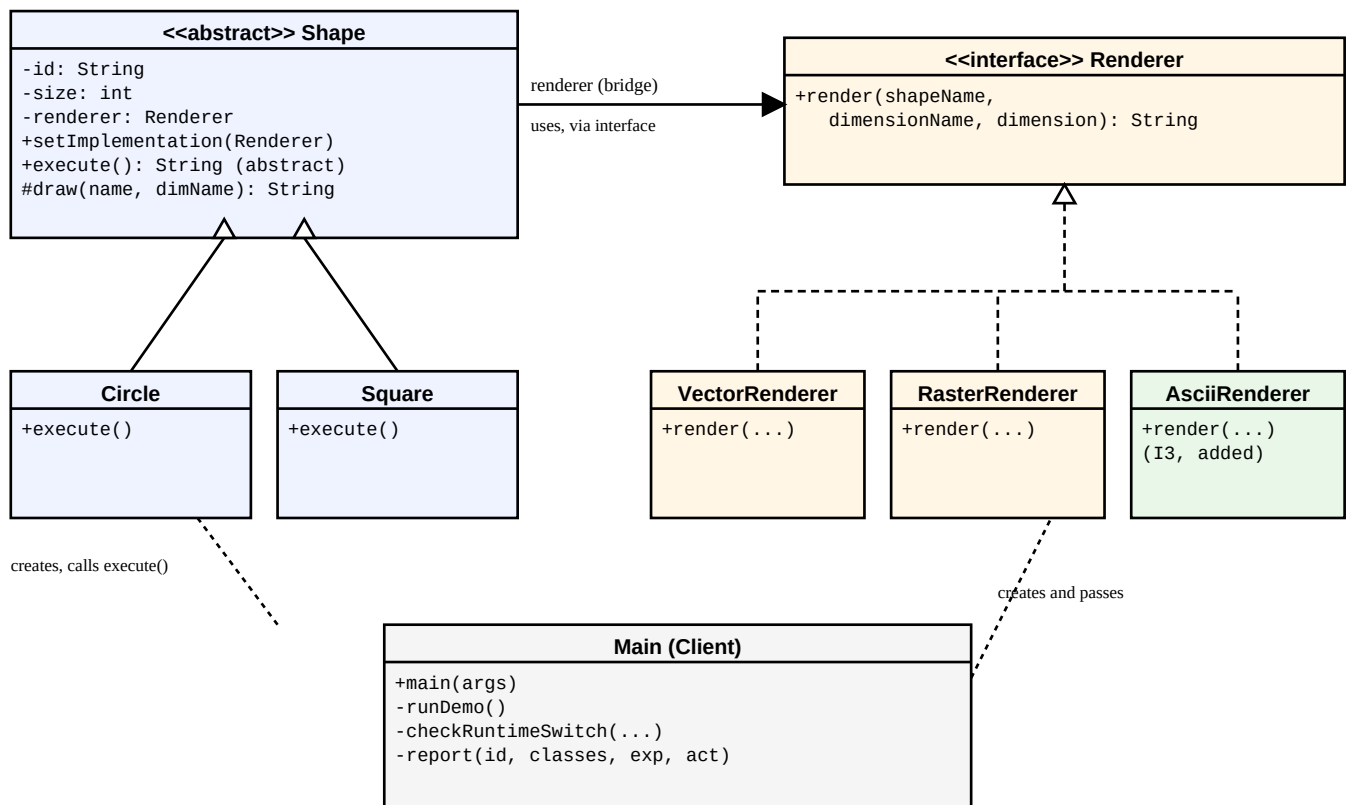
Course: ShP-2216 Software Design Patterns, Astana IT University. **Student:** Tolegen Zhassulan, group SE-2526.

Topic: A (Drawing). **Repository:**

1. Two dimensions and why Bridge

The program has two independent dimensions. The first is *what* is drawn: Shape with Circle (radius 2) and Square (side 3). The second is *how* it is drawn: the Renderer interface with VectorRenderer, RasterRenderer and, after the extension, AsciiRenderer. Without Bridge, every pair needs its own class (CircleVector, CircleRaster, SquareVector...), so the number of classes is shapes times renderers, and each new renderer needs one more class per shape. With Bridge, Shape holds a Renderer reference supplied through the constructor, so the two hierarchies are connected by composition and grow separately: 2 + 3 classes instead of 6.

2. UML class diagram



3. Annotated code excerpts (Clean Code)

a) Separation of responsibilities. Shape only knows its own data and delegates drawing; Renderer only knows how to describe a drawing.

```
protected String draw(String shapeName, String dimensionName) {
    return renderer.render(shapeName, dimensionName, size); // delegation
}
```

Benefit: changing the drawing format never touches the shapes.

b) Meaningful names. Names show the role: `setImplementation`, `RasterRenderer`, `PIXELS_PER_UNIT`, `stateUnchanged`, `sameObject`.

```
private static final int PIXELS_PER_UNIT = 2;
boolean sameObject = original == sameReference;
```

Benefit: the code can be read without comments, and it is easy to explain in the defense.

c) Small focused methods. Each method does one thing, for example `Circle.execute` and `RasterRenderer.render`.

```
public String execute() {
    return draw("circle", "radius");
}
```

Benefit: a method fits in one sentence, so it is easy to test and predict.

d) No duplicated logic. The workflow of calling the renderer is written once in Shape.draw; the check printing is written once in Main.report.

```
// Circle and Square do not repeat renderer.render(...), they call draw(...)
// Main.report(id, classes, expected, actual) is used for T1-T4, T6, T7
```

Benefit: a fix in one place fixes all shapes and all checks.

e) Encapsulation of state. Fields are private and final where possible; outside code reads them through getters and replaces the renderer only through setImplementation.

```
private final String id;
private final int size;
private Renderer renderer;
```

Benefit: id and size cannot be changed by mistake when the implementation is replaced at runtime.

4. Extension, trade-off and Adapter

First the working I1/I2 version was committed (base commit d6b745f73098c7814d2cf81aba94c061d7473001). Then I3 (AsciiRenderer) was added in a second commit. Only the new file src/bridge/AsciiRenderer.java and Main (demo T6 and T7) changed, which is shown in extension.diff. Shape, Circle, Square, Renderer, VectorRenderer and RasterRenderer stayed unchanged, because the Renderer contract was already enough.

Trade-off: Bridge adds more classes and one more level of indirection (a call goes Shape, then Renderer), which is too much for a program with only one variation. Also the Renderer interface must be general enough for all shapes (here: render(shapeName, dimensionName, dimension)).

Bridge vs Adapter: Bridge is planned in advance to let abstraction and implementation vary independently; in my application Shape and Renderer were designed together. Adapter is used afterwards to make an existing incompatible class fit an expected interface, for example if I had to use a third-party drawing library with a different method signature, I would wrap it in an Adapter that implements Renderer, and then plug it into the Bridge.

References

1. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994 (Bridge, Adapter).
2. Refactoring.Guru, Bridge pattern: <https://refactoring.guru/design-patterns/bridge>
3. Martin R. C. Clean Code. Prentice Hall, 2008.
4. Course materials, ShP-2216 Assignment 3 brief.